Alert Interface And File Upload
WebDriver - Lecture - 10

Agenda

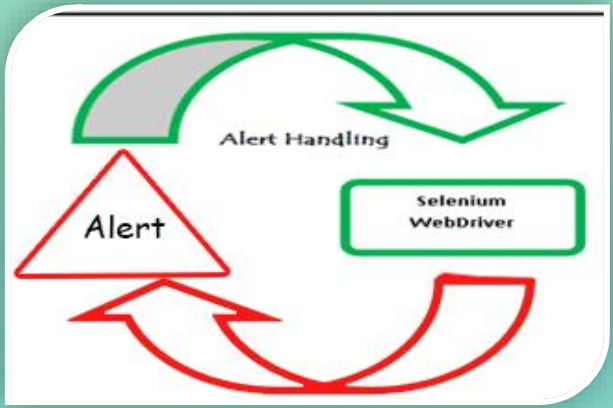
Agenda
Switching to Alert
Alert Interface
File Upload in WebDriver
Upload With Robot Class
Select With sendKeys() Method
WebElement Screenshot
WebStorage in HTML5
Quiz
Homework

Switching to Alert

- **Alert** is a small message box on a **webpage** for displaying:
  - An **on-screen notification** to give the user some kind of information
  - **Permission** to perform a certain kind of operation
  - Some kind of **warning**

- **An Alert**:
  - It is a **separate window** with its own independent **JavaScript** environment.
  - It takes the focus away from the **current window**, and forces the browser to read the message.

- **Switching to Alert**: The changed focus of browser to **Alert** requires switching to it through **TargetLocator** interface and its **alert()** method.




```
// Switching to Alert
Alert alert = driver.switchTo().alert();
```

- **Alert alert()** – This method of **TargetLocator** interface:
  - **Switches** to the currently **active modal dialog** for this particular driver instance
  - **Returns** an instance of **Alert** Interface.

- if an **alert** is present on the webpage and you try to access any of the element in the underlying page you will get exception: **UnhandledAlertException**

- Also if **alert** is not present and we deliberately try to switch to it, **webdriver** throws **NoAlertPresentException**

Alert Interface

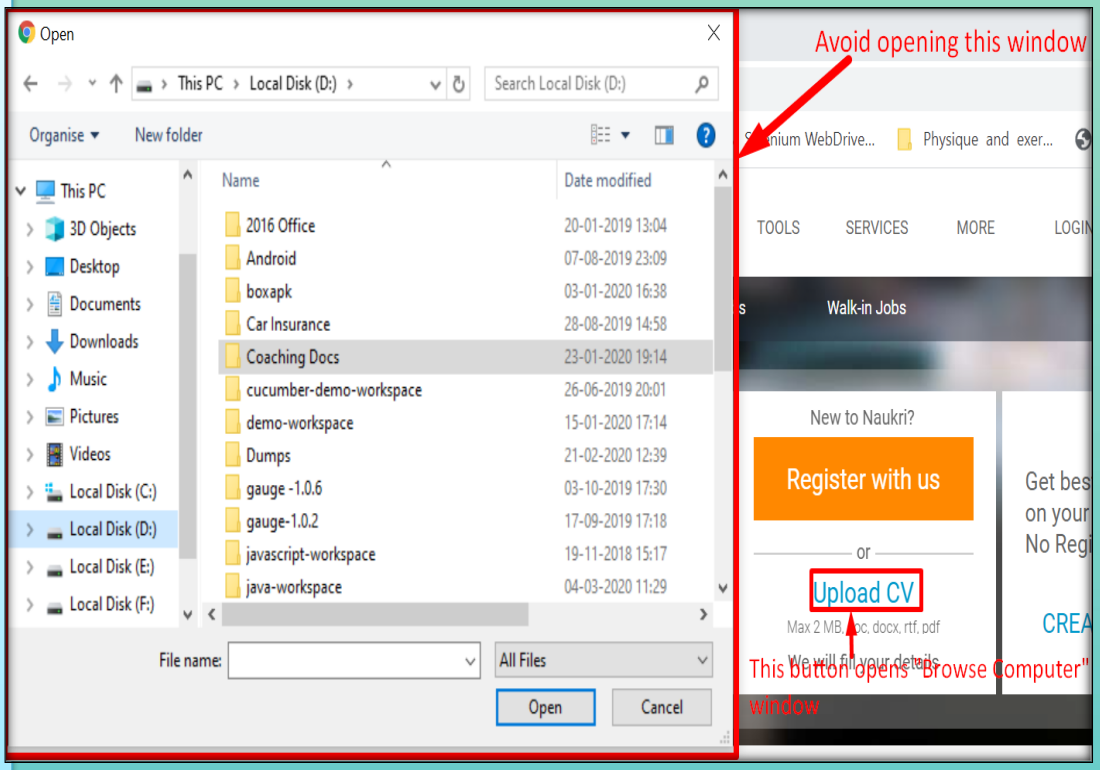
- To handle **independent javascript** alerts, **WebDriver API** provides **Alert** interface.

- Actually the line of code **"driver.switchTO().alert()"** returns an instance of **RemoteAlert** class which is a **private class** inside **"RemoteWebDriver"** class. **RemoteAlert** class further implements **Alert** interface.

- **Methods Of Alert interface:**
  - **void accept ()** - Click on "**OK**" or "**CONFRIM**" button to accept the **Alert**
  - **void dismiss()** - Click on "**CANCEL**" button to dismiss the **Alert**
  - **String getText()** - To get the **text** which is present on the **Alert**
  - **void sendKeys(String keysToSend)** - To enter the text into the **Alert** box

File Upload in WebDriver

- Uploading files in **WebDriver** is done by simply using the **sendKeys()** method on the **file-type input field** and enter the path of the file to be uploaded.
- Find locator for the element "**input with type file**" in the **HTML DOM** which is responsible for opening "**Browse Computer**" window and create its **WebElement** as shown in snippet below
- You should not open "**Browse Computer**" window because it is the native window of Operating system components.
- Rather you should use **sendKeys()** method with upload **WebElement**.
- There is no need to simulate the clicking of the "**Browse**" button. **WebDriver** automatically enters the file path onto the file-selection text box of the **<input type="file">** element
- When setting the file path in **Java**, use the proper escape character for the **back-slash**.



HTML
```
<input type="file" id="file_upload">
```

Snippet
```
WebElement uploadElement = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("input[id='file_upload']")));
uploadElement.sendKeys("D:\\Coaching Docs\\appium_android_automation.pdf");
```

Upload With Robot Class

- When **Selenium Webdriver** cannot handle the **Operating System Window,** at this point **Robot** class in "**java.awt**" package comes handy.

- **Robot** Class can simulate Keyboard and Mouse Event
  - For Example, when upload is **not working** with **sendKeys()** method.

- Uploading File using "**java.awt.Robot**" class:
  Step-1: We have to copy the file location in system clipboard with **StringSelection** and **ToolKit** class.
  Step-2: We have to click on upload button
  Step-3: Simulate pressing **Ctrl+V** and **ENTER** keyboard keys using **Robot** class methods:
  - ***keyPress*** - takes **keyCode** as parameter and presses a given key
  - ***keyRelease*** - takes **keyCode** as parameter and releases a given key

```java
//copy the file path in system clipboard
    StringSelection strSel = new StringSelection("D:\\Coaching Docs\\java basics.txt");
    Toolkit.getDefaultToolkit().getSystemClipboard().setContents(strSel, null);

//upload button click
WebElement uploadElement = wait.until(ExpectedConditions
    .presenceOfElementLocated(By.xpath("//li[contains(@class,'UploadMenuItem')]//span[text()='File']")));
uploadElement.click();

// Create object of Robot class
Robot robo= new Robot();

// Press CTRL+V
robo.keyPress(KeyEvent.VK_CONTROL);
robo.keyPress(KeyEvent.VK_V);


// Release CTRL
robo.keyRelease(KeyEvent.VK_CONTROL);
Thread.sleep(2000);

// Press Enter
robo.keyPress(KeyEvent.VK_ENTER);

// Release Enter
robo.keyRelease(KeyEvent.VK_ENTER);
```

Select With sendKeys() Method

- **Selenium-WebDriver** provides a very robust solution to choose **option** for dropdown with **<select>** tag

- There is no need to create **Select** class for choosing its **option** and it can be achieved by:
  - Create **WebElement** of Select tag.
  - Use **sendKeys()** method of **WebElement** and give argument as "**option name**"

- **Select Option** can be selected with **sendKeys()** method of **WebElement** and giving name of option as parameter.

```
//choosing option in <select> tag with sendKeys() method.
WebElement selectElement = wait.until(ExpectedConditions.
        visibilityOfElementLocated(By.cssSelector("select[name='invite-permission']")));
selectElement.sendKeys("Viewer");
```

WebElement Screenshot

- **Selenium WebDriver** can not take the **screenshot** of **WebElement** because browser has **no javascript method** to take the screenshot.

- **Browsers** may support the method to take a screenshot of the element in the **future**, and we *may able to use* it with the **inbuilt method.**

```
//Locate Image element to capture a screenshot.
WebElement element = wait.until(ExpectedConditions.
        presenceOfElementLocated(By.cssSelector("a.header-logo")));

//Capture entire page screenshot as File.
//Used TakesScreenshot, OutputType Interface of selenium and File class of java to capture a screenshot of the entire page.
File screen = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);

//Used selenium Point class to get x y coordinates of Image element.
//get location(x y coordinates) of the element.
Point point = element.getLocation();
int xcordinate = point.getX();
int ycordinate = point.getY();

//Used selenium getSize() method to get height and width of element.
//Retrieve width of element.
int imageWidth = xcordinate + element.getSize().getWidth();
//Retrieve height of element.
int imageHeight = ycordinate + element.getSize().getHeight();

//Reading full image screenshot.
BufferedImage img = ImageIO.read(screen);

//cut Image using height, width and x y coordinates parameters.
BufferedImage destination = img.getSubimage(xcordinate, ycordinate, imageWidth, imageHeight);
ImageIO.write(destination, "png", screen);

String screenshotPath = System.getProperty("user.dir")+"/screenshots/screenshot_element"+System.currentTimeMillis() + ".png";

//save Image screenshot In project's location
FileUtils.copyFile(screen, new File(screenshotPath));
```

- The alternative steps to take **WebElement's** screenshot:
  Step-1: Locate the **WebElement** to capture **screenshot**
  Step-2: Capture the **screenshot** of entire page as "**File**" output type
  Step-3: Use selenium Point class to get "**x**" and "**y**" coordinates of element
  Step 4: Calculate **sub-image width** which is equal to **"x"** coordinate **plus width** of the element
  Step-5: Calculate **sub-image height** which is equal to "**y**" coordinate **plus height** of the element
  Step-6: Read the **entire page** screenshot to create **full page** image
  Step-7: Cut the **sub-image** from the **full-page** image using **"x"** and **"y"** coordinates and **sub-image width** and **sub-image height**
  Step-8: Save the **WebElement's** screenshot at the project's location

WebStorage in HTML5

- **Selenium WebDriver** provides **WebStorage** interface which provides a way for your web applications to **store data** locally within the user's browser.

- **WebStorage** interface is in "**org.openqa.selenium.html5**" package.

- **Types Of WebStorage:**
  - **Local Storage**
  - **Session Storage**

- **WebStorage** is not useful because it is per origin (**per domain and protocol**) which means if webpage is changed (**url changes**) then this storage expires because all different pages have their own storage data.

- **WebStorage** interface provides two methods:
  - **LocalStorage getLocalStorage()** – This method returns instance of LocalStorage interface
  - **SessionStorage getSessionStorage()** – This method returns instance of SessionStorage interface

Create instance of WebStorage

Set LocalStorage and SessionStorage

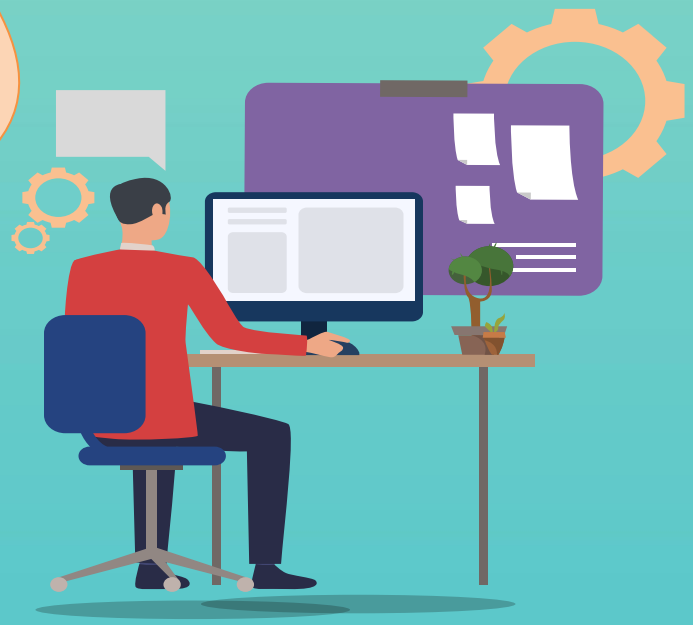
Get LocalStorage and SessionStorage

```
WebStorage webStorage = (WebStorage) driver;
webStorage.getSessionStorage().setItem("name", "qamentor");
webStorage.getLocalStorage().setItem("class", "software testing");

System.out.println("Session Storage "+webStorage.getSessionStorage().getItem("name"));
System.out.println("Local Storage "+webStorage.getLocalStorage().getItem("class"));
```

Quiz

- What is **Alert** on webpage and how can you **switch** to it?
- Can you name **4** methods of **Alert** interface?
- Which **HTML component** is needed for **File Upload**?
- Which method of **WebElement** can be used to do **File Upload** and what type of **argument** is given?
- What is **Robot class** and how is it useful in **WebDriver** scripts?
- Why **Web Storage** is not useful in **WebDriver** scripts?



Homework

- Write **WebDriver** script to upload pdf.
  - Navigate to https://www.docdroid.net/
  - Click on **"Login"** button.
  - Login with username **qacult_demo** and password **qacult12345.**
  - **Upload** any **pdf** on the website.
  - Capture a screenshot of the page as a final proof.

- Create another WebDriver script:
  - Navigate to http://app.box.com
  - Save a **Cookie** with name "**role**".
  - Save a **SessionStorage** with name "**user**". Now **login** into app.
  - After login successfully, **print** on console both saved **Cookie** and **SessionStorage.**
  - Observe the **output**.